

# Estructures de dades lineals

Departament de Ciències de la Computació

Universitat Politècnica de Catalunya



UNIVERSITAT POLITÈCNICA DE CATALUNYA  
BARCELONATECH

Escola Politècnica Superior d'Enginyeria  
de Vilanova i la Geltrú

# Índex

- 1 Tipus genèrics de dades
- 2 Estructures de dades lineals
  - Piles
  - Cues
  - Llistes
- 3 Exercicis

# Índex

- 1 Tipus genèrics de dades
- 2 Estructures de dades lineals
- 3 Exercicis

# Templates de C++

## Definició

**Template:** Mecanisme del C++ per permetre que una classe o mètode treballi amb tipus de dades abstractes.

# Templates de C++

## Definició

**Template:** Mecanisme del C++ per permetre que una classe o mètode treballi amb tipus de dades abstractes.

- En el moment en què s'usa la classe o mètode, es concreta (o s'instancia) el tipus de dades amb el que ha de treballar.

# Templates de C++

## Definició

**Template:** Mecanisme del C++ per permetre que una classe o mètode treballi amb tipus de dades abstractes.

- En el moment en què s'usa la classe o mètode, es concreta (o s'instancia) el tipus de dades amb el que ha de treballar.
- *Exemple:* instanciacions de la classe `vector<...>`

```
vector<int>  
vector<Punt>
```

# Templates de C++

## Definició

**Template:** Mecanisme del C++ per permetre que una classe o mètode treballi amb tipus de dades abstractes.

- En el moment en què s'usa la classe o mètode, es concreta (o s'instancia) el tipus de dades amb el que ha de treballar.
- *Exemple:* instanciacions de la classe `vector<...>`

```
vector<int>  
vector<Punt>
```

- El tipus de dades que contindrà és un paràmetre, per això s'anomenen tipus genèrics o **parametritzats** de dades.

# Tipus genèrics o parametritzats

- Al treballar amb tipus genèrics, els seus mètodes també han de ser-ho  $\Rightarrow$  no poden dependre de la classe dels seus elements.



# Tipus genèrics o parametritzats

- Al treballar amb tipus genèrics, els seus mètodes també han de ser-ho  $\Rightarrow$  no poden dependre de la classe dels seus elements. P.e., no podem fer L/E dels seus elements. ✗

# Tipus genèrics o parametritzats

- Al treballar amb tipus genèrics, els seus mètodes també han de ser-ho  $\Rightarrow$  no poden dependre de la classe dels seus elements. P.e., no podem fer L/E dels seus elements. ✗
- Haurem d'implementar operacions de L/E **fora** de la classe genèrica. Com que no seran de la classe, no tindran paràmetre implícit.

# Tipus genèrics o parametritzats

- Al treballar amb tipus genèrics, els seus mètodes també han de ser-ho  $\Rightarrow$  no poden dependre de la classe dels seus elements. P.e., no podem fer L/E dels seus elements. ✗
- Haurem d'implementar operacions de L/E **fora** de la classe genèrica. Com que no seran de la classe, no tindran paràmetre implícit.
- Per cada classe d'elements, definirem operacions de L/E.

## Tipus genèrics o parametritzats

- Al treballar amb tipus genèrics, els seus mètodes també han de ser-ho  $\Rightarrow$  no poden dependre de la classe dels seus elements. P.e., no podem fer L/E dels seus elements. ✗
- Haurem d'implementar operacions de L/E **fora** de la classe genèrica. Com que no seran de la classe, no tindran paràmetre implícit.
- Per cada classe d'elements, definirem operacions de L/E.
- Per ara, no implementarem tipus de dades genèrics però els farem servir: `stack<...>`, `queue<...>` i `list<...>`.

# Índex

- 1 Tipus genèrics de dades
- 2 Estructures de dades lineals
  - Piles
  - Cues
  - Llistes
- 3 Exercicis

# Estructures de dades lineals

## Definició

**Estructura de dades lineal:** Si els seus elements formen una seqüència.

$$e_1, e_2, \dots, e_n$$

on  $n \geq 0$

# Estructures de dades lineals

## Definició

**Estructura de dades lineal:** Si els seus elements formen una seqüència.

$$e_1, e_2, \dots, e_n$$

on  $n \geq 0$

- $n = 0$ : estructura buida.

# Estructures de dades lineals

## Definició

**Estructura de dades lineal:** Si els seus elements formen una seqüència.

$$e_1, e_2, \dots, e_n$$

on  $n \geq 0$

- $n = 0$ : estructura buida.
- $n = 1$ : té un element (primer i últim), no té predecessor ni successor.



# Estructures de dades lineals

## Definició

**Estructura de dades lineal:** Si els seus elements formen una seqüència.

$$e_1, e_2, \dots, e_n$$

on  $n \geq 0$

- $n = 0$ : estructura buida.
- $n = 1$ : té un element (primer i últim), no té predecessor ni successor.
- $n = 2$ : té dos elements, el primer no té predecessor però sí successor (el segon), el segon no té successor però sí predecessor (el primer).

# Estructures de dades lineals

## Definició

**Estructura de dades lineal:** Si els seus elements formen una seqüència.

$$e_1, e_2, \dots, e_n$$

on  $n \geq 0$

- $n = 0$ : estructura buida.
- $n = 1$ : té un element (primer i últim), no té predecessor ni successor.
- $n = 2$ : té dos elements, el primer no té predecessor però sí successor (el segon), el segon no té successor però sí predecessor (el primer).
- $n > 2$ : tots els elements del segon al penúltim tenen antecessor i successor.

# Estructures de dades lineals

Es diferencien en com es pot accedir als seus elements:

- piles: `stack<...> p;` accés LIFO (Last In, First Out)

# Estructures de dades lineals

Es diferencien en com es pot accedir als seus elements:

- piles: `stack<...> p;` accés LIFO (Last In, First Out)
- cues: `queue<...> q;` accés FIFO (First In, First Out)

# Estructures de dades lineals

Es diferencien en com es pot accedir als seus elements:

- piles: `stack<...> p;` accés LIFO (Last In, First Out)
- cues: `queue<...> q;` accés FIFO (First In, First Out)
- llistes: `list<...> l;` accés a qualsevol element sense treure els anteriors (ús d'iteradors)

# Estructures de dades lineals

Es diferencien en com es pot accedir als seus elements:

- piles: `stack<...> p;` accés LIFO (Last In, First Out)
- cues: `queue<...> q;` accés FIFO (First In, First Out)
- llistes: `list<...> l;` accés a qualsevol element sense treure els anteriors (ús d'iteradors)
- vectors: `vector<...> v;` accés a qualsevol element sense treure els anteriors (ús d'índexs) → ja vistos a FOPR

# Índex

1 Tipus genèrics de dades

2 Estructures de dades lineals

- Piles
- Cues
- Llistes

3 Exercicis

# Classe genèrica `stack` (o `Pila`)

- S'assembla a una pila normal, p.e. una pila de plats.



# Classe genèrica `stack` (o `Pila`)

- S'assembla a una pila normal, p.e. una pila de plats.
- També s'hi assemblen les operacions bàsiques: empilar (`push`), desempilar (`pop`).

## Classe genèrica `stack` (o `Pila`)

- S'assembla a una pila normal, p.e. una pila de plats.
- També s'hi assemblen les operacions bàsiques: empilar (`push`), desempilar (`pop`).
- Accés LIFO: el darrer en arribar és el primer en sortir (Last In, First Out).

# Classe genèrica `stack` (o `Pila`)

- S'assembla a una pila normal, p.e. una pila de plats.
- També s'hi assemblen les operacions bàsiques: empilar (`push`), desempilar (`pop`).
- Accés LIFO: el darrer en arribar és el primer en sortir (Last In, First Out).
- *Exemple:*

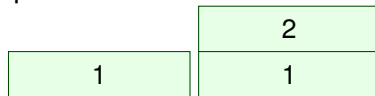
# Classe genèrica `stack` (o `Pila`)

- S'assembla a una pila normal, p.e. una pila de plats.
- També s'hi assemblen les operacions bàsiques: empilar (`push`), desempilar (`pop`).
- Accés LIFO: el darrer en arribar és el primer en sortir (Last In, First Out).
- *Exemple:* empilem 1

A diagram representing a stack. It consists of a single light green rectangular box with a thin black border. Inside the box, the number '1' is centered in black text, representing the element currently at the top of the stack.

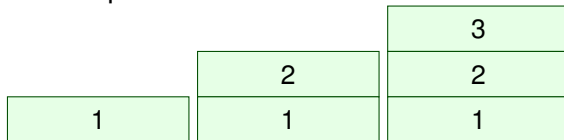
# Classe genèrica `stack` (o `Pila`)

- S'assembla a una pila normal, p.e. una pila de plats.
- També s'hi assemblen les operacions bàsiques: empilar (`push`), desempilar (`pop`).
- Accés LIFO: el darrer en arribar és el primer en sortir (Last In, First Out).
- *Exemple:* ara empilem 2



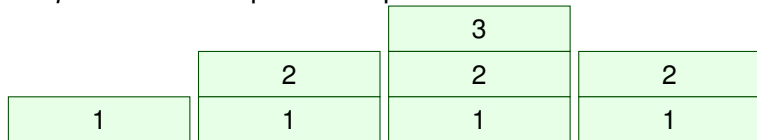
# Classe genèrica `stack` (o Pila)

- S'assembla a una pila normal, p.e. una pila de plats.
- També s'hi assemblen les operacions bàsiques: empilar (`push`), desempilar (`pop`).
- Accés LIFO: el darrer en arribar és el primer en sortir (Last In, First Out).
- *Exemple:* ara empilem 3



# Classe genèrica `stack` (o Pila)

- S'assembla a una pila normal, p.e. una pila de plats.
- També s'hi assemblen les operacions bàsiques: empilar (`push`), desempilar (`pop`).
- Accés LIFO: el darrer en arribar és el primer en sortir (Last In, First Out).
- *Exemple:* ara desempilem 1 cop



# Especificació de la classe `stack` (o Pila)

- A l'especificació de la classe `stack` apareix la paraula reservada `template`.

```
template <class T> class stack
```



# Especificació de la classe `stack` (o Pila)

- A l'especificació de la classe `stack` apareix la paraula reservada `template`.

```
template <class T> class stack
```

- `T` fa referència al tipus dels elements que contindrà la pila, que pot ser qualsevol.

# Especificació de la classe `stack` (o Pila)

- A l'especificació de la classe `stack` apareix la paraula reservada `template`.

```
template <class T> class stack
```

- `T` fa referència al tipus dels elements que contindrà la pila, que pot ser qualsevol.
- A l'instanciar, substituïm `T` pel tipus que vulguem.

*Exemples:*

# Especificació de la classe `stack` (o Pila)

- A l'especificació de la classe `stack` apareix la paraula reservada `template`.

```
template <class T> class stack
```

- `T` fa referència al tipus dels elements que contindrà la pila, que pot ser qualsevol.
- A l'instanciar, substituïm `T` pel tipus que vulguem.

*Exemples:*

```
stack<int> pi;
```

```
stack<Punt> p;
```

# Especificació de la classe `stack` (o Pila)

```
template <class T> class stack {  
    // Tipus de mòdul: dades  
    // Descripció del tipus: Estructura lineal  
    // que conté elements de tipus T, que permet  
    // afegir, consultar i eliminar elements només  
    // en un dels seus extrems.  
private:  
  
public:  
    // Constructors i destructor  
    // Modificadors  
    // Consultors  
};
```

# Constructors i destructor de `stack`

```
stack();  
/* Pre: cert */  
/* Post: Construeix una pila buida */  
  
stack(const stack &orig);  
/* Pre: cert */  
/* Post: Construeix una pila que és una còpia d'orig */  
  
// Esborra automàticament els objectes  
// locals en sortir d'un àmbit de visibilitat  
~stack();
```

# Modificadors de stack

```
void push(const T &x);  
/* Pre: El paràmetre implícit és P */  
/* Post: El paràmetre implícit és p , on p és igual a P  
    amb x afegit com a darrer element (en el cim) */  
  
void pop();  
/* Pre: El paràmetre implícit és P i P no és una pila  
    buida */  
/* Post: El paràmetre implícit és p , on p és igual a P  
    sense el darrer element afegit (el que estava en el  
    cim) */
```

# Consultors de stack

```
T top() const;
/* Pre: El paràmetre implícit no és una pila buida */
/* Post: El resultat és el darrer valor afegit al p.i.,
   i.e. l'element situat al cim de la pila */

bool empty() const;
/* Pre: cert */
/* Post: El resultat indica si el paràmetre implícit és
   una pila buida */

int size() const;
/* Pre: cert */
/* Post: El resultat és el nombre d'elements del paràmetre
   implícit */
```

# Ús de la classe `stack`

- Farem exercicis senzills d'ús de la classe `stack`.



# Ús de la classe `stack`

- Farem exercicis senzills d'ús de la classe `stack`.
- Codificarem només les accions i funcions en C++, no el `main`.

# Ús de la classe `stack`

- Farem exercicis senzills d'ús de la classe `stack`.
- Codificarem només les accions i funcions en C++, no el `main`.
- Quan passem les piles per referència:
  - Evitem que faci una còpia del paràmetre a cada crida recursiva (programes recursius).

# Ús de la classe `stack`

- Farem exercicis senzills d'ús de la classe `stack`.
- Codificarem només les accions i funcions en C++, no el `main`.
- Quan passem les piles per referència:
  - Evitem que faci una còpia del paràmetre a cada crida recursiva (programes recursius).
  - Estalvi de temps i de memòria.

# Ús de la classe `stack`

- Farem exercicis senzills d'ús de la classe `stack`.
- Codificarem només les accions i funcions en C++, no el `main`.
- Quan passem les piles per referència:
  - Evitem que faci una còpia del paràmetre a cada crida recursiva (programes recursius).
  - Estalvi de temps i de memòria.
  - Si volem mantenir la pila original, cal fer còpia abans de cridar a la funció.

# Exemple 1: suma dels elements d'una pila de `int`

Especificació de la funció:

```
int suma_pila_int(stack<int> p);  
/* Pre: p = P */  
/* Post: El resultat és la suma dels elements de P */
```

# Solució ex.1: suma dels elements d'una pila de `int`

# Solució ex.1: suma dels elements d'una pila de `int`

```
int suma_pila_int(stack<int> p)
/* Pre: p = P */
/* Post: El resultat és la suma dels elements de P */
{
    int s = 0;
    while (not p.empty()) {
        s += p.top();
        p.pop();
    }
    return s;
}
```

## Solució ex.1: suma dels elements d'una pila de `int`

```
int suma_pila_int(stack<int> p)
/* Pre: p = P */
/* Post: El resultat és la suma dels elements de P */
{
    int s = 0;
    while (not p.empty()) {
        s += p.top();
        p.pop();
    }
    return s;
}
```

Pregunta: Sabríeu escriure la funció de forma **recursiva**?



## Exemple 2: sumar $k$ als elements d'una pila de `int`

Especificació de la funció:

```
stack<int> sumar_k_pila(stack<int> p, int k);  
/* Pre: p = P */  
/* Post: Cada element del resultat és la suma de  
        l'element de P que ocupa la mateixa posició més  
        el valor k */
```

## Solució ex.2: sumar $k$ als elements d'una pila de `int`

```
stack<int> sumar_k_pila(stack<int> p, int k)
/* Pre: p = P */
/* Post: Cada element del resultat és la suma de l'element
de P que ocupa la mateixa posició més el valor k */
{
    stack<int> paux;
    while (not p.empty()) {
        int elem = p.top() + k;
        p.pop();
        paux.push(elem);
    } // p ha quedat buida, paux conté P+k invertida
    while (not paux.empty()) {
        p.push(paux.top());
        paux.pop();
    }
    return p;
}
```

Nota: La versió recursiva no requereix donar la volta a la pila.

# Índex

1 Tipus genèrics de dades

2 Estructures de dades lineals

- Piles
- Cues
- Llistes

3 Exercicis

# Classe genèrica `queue` (o `Cua`)

- S'assembla a una cua normal, p.e. una cua de persones.

# Classe genèrica `queue` (o `Cua`)

- S'assembla a una cua normal, p.e. una cua de persones.
- També s'hi assemblen les operacions bàsiques: demanar tanda/torn (`push`), avançar (`pop`).

# Classe genèrica `queue` (o `Cua`)

- S'assembla a una cua normal, p.e. una cua de persones.
- També s'hi assemblen les operacions bàsiques: demanar tanda/torn (`push`), avançar (`pop`).
- Accés FIFO: el primer en arribar és el primer en sortir (First In, First Out).

# Classe genèrica `queue` (o `Cua`)

- S'assembla a una cua normal, p.e. una cua de persones.
- També s'hi assemblen les operacions bàsiques: demanar tanda/torn (`push`), avançar (`pop`).
- Accés FIFO: el primer en arribar és el primer en sortir (First In, First Out).
- *Exemple:*

# Classe genèrica `queue` (o `Cua`)

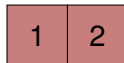
- S'assembla a una cua normal, p.e. una cua de persones.
- També s'hi assemblen les operacions bàsiques: demanar tanda/torn (`push`), avançar (`pop`).
- Accés FIFO: el primer en arribar és el primer en sortir (First In, First Out).
- *Exemple:* 1 demana tanda





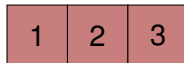
# Classe genèrica `queue` (o `Cua`)

- S'assembla a una cua normal, p.e. una cua de persones.
- També s'hi assemblen les operacions bàsiques: demanar tanda/torn (`push`), avançar (`pop`).
- Accés FIFO: el primer en arribar és el primer en sortir (First In, First Out).
- *Exemple:* ara 2 demana tanda



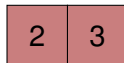
# Classe genèrica `queue` (o `Cua`)

- S'assembla a una cua normal, p.e. una cua de persones.
- També s'hi assemblen les operacions bàsiques: demanar tanda/torn (`push`), avançar (`pop`).
- Accés FIFO: el primer en arribar és el primer en sortir (First In, First Out).
- *Exemple:* ara 3 demana tanda



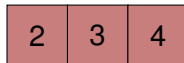
# Classe genèrica `queue` (o `Cua`)

- S'assembla a una cua normal, p.e. una cua de persones.
- També s'hi assemblen les operacions bàsiques: demanar tanda/torn (`push`), avançar (`pop`).
- Accés FIFO: el primer en arribar és el primer en sortir (First In, First Out).
- *Exemple:* ara avança la cua



# Classe genèrica `queue` (o `Cua`)

- S'assembla a una cua normal, p.e. una cua de persones.
- També s'hi assemblen les operacions bàsiques: demanar tanda/torn (`push`), avançar (`pop`).
- Accés FIFO: el primer en arribar és el primer en sortir (First In, First Out).
- *Exemple:* ara 4 demana tanda



# Especificació de la classe `queue` (o `Cua`)

- A l'especificació de la classe `queue` apareix la paraula reservada `template`.

```
template <class T> class queue
```

# Especificació de la classe `queue` (o `Cua`)

- A l'especificació de la classe `queue` apareix la paraula reservada `template`.

```
template <class T> class queue
```

- `T` fa referència al tipus dels elements que contindrà la cua, que pot ser qualsevol.

# Especificació de la classe `queue` (o Cua)

- A l'especificació de la classe `queue` apareix la paraula reservada `template`.

```
template <class T> class queue
```

- `T` fa referència al tipus dels elements que contindrà la cua, que pot ser qualsevol.
- A l'instanciar, substituïm `T` pel tipus que vulguem.

*Exemples:*

# Especificació de la classe `queue` (o Cua)

- A l'especificació de la classe `queue` apareix la paraula reservada `template`.

```
template <class T> class queue
```

- `T` fa referència al tipus dels elements que contindrà la cua, que pot ser qualsevol.
- A l'instanciar, substituïm `T` pel tipus que vulguem.

*Exemples:*

```
queue<int> c;  
queue<Punt> cp;
```



# Especificació de la classe queue (o Cua)

```
template <class T> class queue {  
    // Tipus de mòdul: dades  
    // Descripció del tipus: Estructura lineal  
    // que conté elements de tipus T, que permet  
    // afegir elements només al final, i consultar i  
    // eliminar només el seu primer element.  
  
private:  
  
public:  
    // Constructors i destructor  
    // Modificadors  
    // Consultors  
};
```

# Constructors i destructor de queue

```
queue();  
/* Pre: cert */  
/* Post: Construeix una cua buida */  
  
queue(const queue &orig);  
/* Pre: cert */  
/* Post: Construeix una cua que és una còpia d'orig */  
  
// Esborra automàticament els objectes  
// locals en sortir d'un àmbit de visibilitat  
~queue();
```

# Modificadors de queue

```
void push(const T &x);
```

```
/* Pre: El paràmetre implícit és P */
```

```
/* Post: El paràmetre implícit és p, on p és igual a P amb  
x afegit com a darrer element */
```

```
void pop();
```

```
/* Pre: El paràmetre implícit és P i P no és una cua buida */
```

```
/* Post: El paràmetre implícit és p, on p és igual a P sense  
el primer element */
```

# Consultors de queue

```
T front() const;
/* Pre: El paràmetre implícit no és una cua buida */
/* Post: El resultat és el primer element del paràmetre
    implícit */

bool empty() const;
/* Pre: cert */
/* Post: El resultat indica si el paràmetre implícit és una
    cua buida */

int size() const;
/* Pre: cert */
/* Post: El resultat és el nombre d'elements del paràmetre
    implícit */
```

## Exemple 3: suma dels elements d'una cua

Especificació de la funció:

```
int suma_cua_int(queue<int> c);  
/* Pre: c = C */  
/* Post: El resultat és la suma dels elements de C */
```

# Solució ex.3: suma dels elements d'una cua

## Solució ex.3: suma dels elements d'una cua

```
int suma_cua_int(queue<int> c)
/* Pre: c = C */
/* Post: El resultat és la suma dels elements de C */
{
    int s = 0;
    while (not c.empty()) {
        s += c.front();
        c.pop();
    }
    return s;
}
```

# Exercici 1: sumar $k$ als elements d'una cua

Especificació de la funció:

```
void sumar_k_cua(queue<int> &c, int k);  
/* Pre: c = C */  
/* Post: Cada element de c és la suma del valor de k i de  
l'element de C a la mateixa posició */
```



# Índex

1 Tipus genèrics de dades

2 Estructures de dades lineals

- Piles
- Cues
- Llistes

3 Exercicis

# Llistes i iteradors (`list` i `iterator`)

## Definició

**Contenidor:** Estructura de dades on s'emmagatzemen objectes d'una forma determinada.

# Llistes i iteradors (`list` i `iterator`)

## Definició

**Contenidor:** Estructura de dades on s'emmagatzemen objectes d'una forma determinada.

- *Exemples STL:* `vector<T>`, `stack<T>`, `queue<T>`

# Llistes i iteradors (`list` i `iterator`)

## Definició

**Contenidor:** Estructura de dades on s'emmagatzemen objectes d'una forma determinada.

- *Exemples STL:* `vector<T>`, `stack<T>`, `queue<T>`
- Una **llista** és una estructura de dades lineal que permet l'accés a qualsevol element **sense haver de treure els anteriors**.

# Llistes i iteradors (`list` i `iterator`)

## Definició

**Contenidor:** Estructura de dades on s'emmagatzemen objectes d'una forma determinada.

- *Exemples STL:* `vector<T>`, `stack<T>`, `queue<T>`
- Una **llista** és una estructura de dades lineal que permet l'accés a qualsevol element **sense haver de treure els anteriors**.
- Un **iterador** ens permet desplaçar-nos per un contenidor i fer referència als seus elements. Existeixen diferents tipus d'iteradors depenent de les característiques del contenidor.

# Llistes i iteradors (`list` i `iterator`)

## Definició

**Contenidor:** Estructura de dades on s'emmagatzemen objectes d'una forma determinada.

- *Exemples STL:* `vector<T>`, `stack<T>`, `queue<T>`
- Una **llista** és una estructura de dades lineal que permet l'accés a qualsevol element **sense haver de treure els anteriors**.
- Un **iterador** ens permet desplaçar-nos per un contenidor i fer referència als seus elements. Existeixen diferents tipus d'iteradors depenent de les característiques del contenidor.
- Per exemple, per les llistes d'enters, declararem una llista i un iterador així:

```
list<int> l;  
list<int>::iterator it;
```

# Llistes i iteradors

- Un iterador no s'associa a un contenidor sinó a una classe de contenidors  $\Rightarrow$  pot servir per més d'un contenidor alhora.

# Llistes i iteradors

- Un iterador no s'associa a un contenidor sinó a una classe de contenidors  $\Rightarrow$  pot servir per més d'un contenidor alhora.
- Mètode `begin()`: retorna un iterador que referencia el primer element del contenidor, si existeix.



# Llistes i iteradors

- Un iterador no s'associa a un contenidor sinó a una classe de contenidors  $\Rightarrow$  pot servir per més d'un contenidor alhora.
- Mètode `begin()`: retorna un iterador que referencia el primer element del contenidor, si existeix. *Exemple:*

```
list<Punt>::iterator it = l.begin();
```

- `it` referencia el primer element de `l`.

# Llistes i iteradors

- Un iterador no s'associa a un contenidor sinó a una classe de contenidors  $\Rightarrow$  pot servir per més d'un contenidor alhora.
- Mètode `begin()`: retorna un iterador que referencia el primer element del contenidor, si existeix. *Exemple:*

```
list<Punt>::iterator it = l.begin();
```

- `it` referencia el primer element de `l`.
- `it` està associat al contenidor `list<Punt>`.

# Listes i iterators

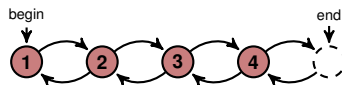
- Un iterator no s'associa a un contenidor sinó a una classe de contenidors  $\Rightarrow$  pot servir per més d'un contenidor alhora.
- Mètode `begin()`: retorna un iterator que referencia el primer element del contenidor, si existeix. *Exemple:*

```
list<Punt>::iterator it = l.begin();
```

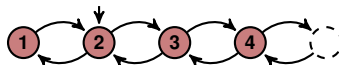
- `it` referencia el primer element de `l`.
- `it` està associat al contenidor `list<Punt>`.
- Mètode `end()`: retorna un iterator que referencia a un element fictici posterior al darrer element del contenidor.

# Evolució d'una llista

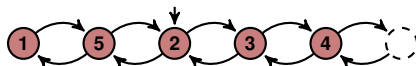
```
it=l.begin();
```



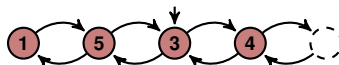
```
++it;
```



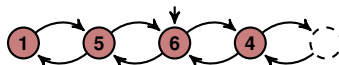
```
l.insert(it, 5);
```



```
it=l.erase(it);
```



```
*it=6;
```



# Tipus d'iteradors i operacions amb iteradors

- **Iteradors constants:** no permeten modificar l'objecte referenciat. Obligatori per recórrer un contenidor que sigui un paràmetre `const`.

# Tipus d'iteradors i operacions amb iteradors

- **Iteradors constants:** no permeten modificar l'objecte referenciat. Obligatori per recórrer un contenidor que sigui un paràmetre `const`.
- *Exemple:* `list<Punt>::const_iterator itp;`

# Tipus d'iteradors i operacions amb iteradors

- **Iteradors constants:** no permeten modificar l'objecte referenciat. Obligatori per recórrer un contenidor que sigui un paràmetre `const`.
- *Exemple:* `list<Punt>::const_iterator itp;`
- Per moure'ns per un contenidor avancem/retrocedim amb un iterador: `++it / --it`.

# Tipus d'iteradors i operacions amb iteradors

- **Iteradors constants:** no permeten modificar l'objecte referenciat. Obligatori per recórrer un contenidor que sigui un paràmetre `const`.
- *Exemple:* `list<Punt>::const_iterator itp;`
- Per moure'ns per un contenidor avancem/retrocedim amb un iterador: `++it` / `--it`.  
Dóna error:
  - Fer `--it` quan es compleix que `it==l.begin()`.



# Tipus d'iteradors i operacions amb iteradors

- **Iteradors constants:** no permeten modificar l'objecte referenciat. Obligatori per recórrer un contenidor que sigui un paràmetre `const`.
- *Exemple:* `list<Punt>::const_iterator itp;`
- Per moure'ns per un contenidor avancem/retrocedim amb un iterador: `++it` / `--it`.  
Dóna error:
  - Fer `--it` quan es compleix que `it==l.begin()`.
  - Fer `++it` quan es compleix que `it==l.end()`.

# Més operacions amb iteradors

- Desreferenciar un iterador és obtenir l'objecte al que referencia. S'usa la notació `*`.

# Més operacions amb iteradors

- Desreferenciar un iterador és obtenir l'objecte al que referencia. S'usa la notació `*`. *Exemple:*
  - `*itp` conté l'objecte de tipus `Punt` referenciat per `itp`.

# Més operacions amb iteradors

- Desreferenciar un iterador és obtenir l'objecte al que referencia. S'usa la notació `*`. *Exemple:*
  - `*itp` conté l'objecte de tipus `Punt` referenciat per `itp`.
  - Podem consultar la seva coordenada `x`:  
`(*itp).coordenadax()`;

# Més operacions amb iteradors

- Desreferenciar un iterador és obtenir l'objecte al que referencia. S'usa la notació `*`. *Exemple:*
  - `*itp` conté l'objecte de tipus `Punt` referenciat per `itp`.
  - Podem consultar la seva coordenada `x`:  
`(*itp).coordenadax()`;
  - No podem modificar `*itp` perquè `itp` és un iterador constant.

# Més operacions amb iteradors

- Desreferenciar un iterador és obtenir l'objecte al que referencia. S'usa la notació `*`. *Exemple:*
  - `*itp` conté l'objecte de tipus `Punt` referenciat per `itp`.
  - Podem consultar la seva coordenada `x`:  
`(*itp).coordenadax()`;
  - No podem modificar `*itp` perquè `itp` és un iterador constant.
  - Dóna error:
    - Desreferenciar un iterador a l'element `end()` d'una llista.

# Més operacions amb iteradors

- Desreferenciar un iterador és obtenir l'objecte al que referencia. S'usa la notació `*`. *Exemple:*
  - `*itp` conté l'objecte de tipus `Punt` referenciat per `itp`.
  - Podem consultar la seva coordenada `x`:  
`(*itp).coordenadaX();`
  - No podem modificar `*itp` perquè `itp` és un iterador constant.
  - Dóna error:
    - Desreferenciar un iterador a l'element `end()` d'una llista.
    - Desreferenciar un iterador no assignat.

# Més operacions amb iteradors

- Desreferenciar un iterador és obtenir l'objecte al que referencia. S'usa la notació `*`. *Exemple:*
  - `*itp` conté l'objecte de tipus `Punt` referenciat per `itp`.
  - Podem consultar la seva coordenada `x`:  
`(*itp).coordenadax()`;
  - No podem modificar `*itp` perquè `itp` és un iterador constant.
  - Dóna error:
    - Desreferenciar un iterador a l'element `end()` d'una llista.
    - Desreferenciar un iterador no assignat.
- Assignació: `it1 = it2`;



# Més operacions amb iteradors

- Desreferenciar un iterador és obtenir l'objecte al que referencia. S'usa la notació `*`. *Exemple:*
  - `*itp` conté l'objecte de tipus `Punt` referenciat per `itp`.
  - Podem consultar la seva coordenada `x`:  
`(*itp).coordenadax()`;
  - No podem modificar `*itp` perquè `itp` és un iterador constant.
  - Dóna error:
    - Desreferenciar un iterador a l'element `end()` d'una llista.
    - Desreferenciar un iterador no assignat.
- Assignació: `it1 = it2`;
- Comparació: `it1 == it2, it1 != it2`

# Més operacions amb iteradors

- Desreferenciar un iterador és obtenir l'objecte al que referencia. S'usa la notació `*`. *Exemple:*
  - `*itp` conté l'objecte de tipus `Punt` referenciat per `itp`.
  - Podem consultar la seva coordenada `x`:  
`(*itp).coordenadaX();`
  - No podem modificar `*itp` perquè `itp` és un iterador constant.
  - Dóna error:
    - Desreferenciar un iterador a l'element `end()` d'una llista.
    - Desreferenciar un iterador no assignat.
- Assignació: `it1 = it2;`
- Comparació: `it1 == it2, it1 != it2`
- Si `l` és una llista amb un iterador `it`,  
`it == l.end()` indica que hem arribat al final del contenidor.

# Especificació de la classe `list` (o `Llista`)

```
template <class T> class list{  
  // Tipus de mòdul: dades  
  // Descripció del tipus: Estructura lineal que conté  
  // elements de tipus T, que es pot començar a consultar  
  // pels extrems, on des de cada element es pot accedir  
  // a l'element anterior i posterior (si existeixen), i  
  // que admet afegir-hi i esborrar-hi elements a qualsevol  
  // punt.  
  
private:  
  
public:  
};
```

# Constructors i destructor de `list`

```
list();  
/* Pre: cert */  
/* Post: El resultat és una llista sense cap element */  
  
list(const list &orig);  
/* Pre: cert */  
/* Post: El resultat és una còpia d'orig */  
  
//Esborra automàticament els objectes  
// locals en sortir d'un àmbit de visibilitat  
~list();
```

# Modificadors de `list`

```
void clear();  
/* Pre: cert */  
/* Post: El paràmetre implícit és una llista buida */  
  
void insert(iterator it, const T &x);  
/* Pre: it referencia algun element existent al paràmetre  
implícit o és igual a l'end() d'aquest */  
/* Post: El paràmetre implícit és com el paràmetre implícit  
original amb x davant de l'element referenciat per it  
al paràmetre implícit original */
```

# Més modificadors de `list`

```
iterator erase(iterator it);
```

```
/* Pre: it referencia algun element existent al paràmetre  
implícit, que no és buit */
```

```
/* Post: El paràmetre implícit és com el paràmetre implícit  
original sense l'element referenciat per l'it original;  
el resultat referencia l'element següent al que  
referenciava it al paràmetre implícit original */
```

```
void splice(iterator it, list &l);
```

```
/* Pre: l=L, it referencia algun element del paràmetre  
implícit o és igual a l'end() d'aquest; el p.i. i l no són  
el mateix objecte */
```

```
/* Post: El paràmetre implícit té els seus elements originals i  
els d'l inserits davant de l'element referenciat per it;  
l està buida */
```

# Consultors de list

```
bool empty() const;
/* Pre: cert */
/* Post: El resultat indica si el paràmetre implícit té
        elements o no */

int size() const;
/* Pre: cert */
/* Post: El resultat és el nombre d'elements del paràmetre
        implícit */
```

## Exemple 4: suma dels elements d'una llista d'enters

Especificació de la funció:

```
int suma_llista_int(const list<int> &l);  
/* Pre: cert */  
/* Post: El resultat és la suma dels elements de l */
```



# Solució ex.4: suma dels elements d'una llista d'enters

## Solució ex.4: suma dels elements d'una llista d'enters

```
int suma_llista_int(const list<int> &l)
/* Pre: cert */
/* Post: El resultat és la suma dels elements de l */
{
    int s = 0;
    for (list<int>::const_iterator it=l.begin(); it!=l.end(); ++it)
    {
        s += *it;
    }
    return s;
}
```

## Exercici 2: sumar $k$ als elements d'una llista

Especificació de la funció:

```
void sumar_k_llista(list<int> &l, int k);  
/* Pre: l = L */  
/* Post: Cada element de l és el valor corresponent de L  
        més k */
```

# Preguntes

- Per crear o modificar una llista podem fer servir iteradors constants?

# Preguntes

- Per crear o modificar una llista podem fer servir iteradors constants?
- Quan una llista `l` és buida, quina relació hi ha entre `l.begin()` i `l.end()`?

# Preguntes

- Per crear o modificar una llista podem fer servir iteradors constants?
- Quan una llista `l` és buida, quina relació hi ha entre `l.begin()` i `l.end()`?
- Quan explorem una llista, la modifiquem? I una pila? I una cua?

# Índex

- 1 Tipus genèrics de dades
- 2 Estructures de dades lineals
- 3 Exercicis**

# Exercicis

- Resoleu els exercicis proposats en aquesta sessió.
- Resoleu exercicis sobre piles, cues i llistes de la col·lecció de problemes.